Design Note

MVP Overview

An LLD (Low-Level Design) practice platform with a tight feedback loop:

Choose Problem → Think/Design → Submit → Get Feedback → Review → Try Again

The platform demonstrates end-to-end: a learner selects a problem, drafts a solution, submits it, receives structured multi-criteria feedback, reviews past attempts, and iterates.

User Flow

1. **Browse Problems** — Learner sees a catalog of 6 LLD problems with titles, descriptions, requirements, constraints, and hints.
2. **Start Attempt** — Learner selects a problem, creating a new `Attempt` in `DRAFT` status.
3. **Design Solution** — Learner writes their solution (class diagrams, pseudocode, or code) and an explanation of design decisions.
4. **Save Draft** — Progress is persisted; the attempt remains in `DRAFT`.
5. **Submit** — Learner submits; status transitions to `FEEDBACK_PENDING`.
6. **Get Feedback** — Feedback is generated by a hybrid engine (rule-based + simulated LLM) and merged into unified scores.
7. **Review** — Learner views detailed criterion-by-criterion scores, comments, and improvement suggestions.
8. **History** — All past attempts are listed, sorted by date, showing status and outcome.
9. **Try Again** — Learner starts a new attempt on the same or a different problem.

Key Classes & Interfaces

Domain Entities (`src/domain/models/`)

Class	Responsibility
`Learner`	Represents a user (id, name, email)
`LldProblem`	Problem definition (title, description, requirements, constraints, hints, tags, difficulty)
`Solution`	The learner's submission (content, format, explanation, timestamps)
`Attempt`	Links learner + problem + solution + status; manages state transitions
`Feedback`	Aggregated scores across criteria, summary, suggestions
`CriterionScore`	A single criterion score (0-5) with comment
`AttemptHistory`	Collection of attempts for history/review

Domain Interfaces (`src/domain/interfaces/`)

Interface	Purpose	Pattern
`FeedbackProvider`	Abstract base for feedback strategies	Strategy
`ProblemRepository`	Port for problem data access	Repository / DIP
`AttemptRepository`	Port for attempt persistence	Repository / DIP
`LearnerRepository`	Port for learner data access	Repository / DIP

Application Services (`src/application/services/`)

Service	Responsibility
`PracticeService`	Orchestrates the practice lifecycle (list, start, draft, submit, history)
`FeedbackService`	Orchestrates feedback generation from multiple providers with graceful fallback

Use Cases (`src/application/use-cases/`)

Individual, single-responsibility use case classes: `ListProblems` , `StartAttempt` , `SaveSolutionDraft` , `SubmitAttempt` , `GenerateFeedback` , `GetAttemptHistory` .

Infrastructure (`src/infrastructure/`)

Component	Purpose
`InMemoryProblemRepository`	In-memory problem store, pre-seeded with catalog
`InMemoryAttemptRepository`	In-memory attempt store with learner/problem queries
`InMemoryLearnerRepository`	In-memory learner store
`RuleBasedFeedbackProvider`	Deterministic scoring via keyword/structural heuristics
`LLMBasedFeedbackProvider`	LLM-based scoring (real OpenAI API or simulation mode)
`FeedbackProviderChain`	Chain of Responsibility for graceful fallback
`problemCatalog.js`	6 curated LLD problems

Evaluation Approach

Deterministic vs. LLM

Aspect	Deterministic (Rule-Based)	LLM-Based
Speed	Fast (ms)	Slower (API latency / simulated 500ms)
Reproducibility	Fully reproducible	May vary by prompt/model
What it checks	Structural: class count, pattern keywords, SOLID keywords, completeness ratio	Qualative: design appropriateness, alternative trade-offs, nuanced feedback
Failure mode	Never fails (pure function)	Can fail (API error, network)

The `FeedbackService` runs **both** providers and **merges** their results, combining deterministic scores with LLM insights. If the LLM provider fails, the rule-based scores still produce feedback (graceful degradation).

Feedback Criteria (7 dimensions, 0–5 each)

1. **Modularity** — Class decomposition quality (class count, method distribution)
2. **SOLID Principles** — Explicit mention/application of SRP, OCP, LSP, ISP, DIP
3. **Design Patterns** — Use of Strategy, Factory, Observer, State, Command, etc.
4. **Responsibilities** — Single responsibility, separation of concerns

5. **Relationships** — Inheritance, composition, aggregation, association modeling
6. **Trade-offs** — Explicit analysis of design alternatives
7. **Completeness** — Coverage of all requirements and constraints

Async & Failure Handling

- **Slow evaluation**: The LLM provider uses `setTimeout` to simulate API latency. In production, this would be an async API call. The feedback endpoint blocks until both providers complete, keeping the architecture simple (no distributed-systems complexity per the scope boundary).
- **Provider failure**: If the LLM provider throws, the `FeedbackService` catches the error, records it, and still returns rule-based feedback. Only if **all** providers fail does the attempt get marked as `FEEDBACK_FAILED`.
- **Client-side**: The frontend shows a "Generating..." spinner and handles errors with user-facing notifications.

Extensibility

Adding a New Feedback Provider

```
// 1. Extend the FeedbackProvider interface
class MyFeedbackProvider extends FeedbackProvider {
  constructor() { super(FeedbackProviderType.LLM); }
  async provideFeedback(attempt, problem) { ... }
}

// 2. Register in container.js
feedbackProviders: { rule, llm: new LLMBasedFeedbackProvider(), custom: new MyFeedba
```

Adding a New Submission Format

```
// 1. Add to enums
const SolutionFormat = Object.freeze({ ..., IMAGE: 'IMAGE' });

// 2. The Solution model already stores format generically
```

Swapping Storage

```
// 1. Implement the interface
class MongoProblemRepository extends ProblemRepository { ... }

// 2. Inject in container.js
this.problemRepository = new MongoProblemRepository();
```

Architecture: Clean / Hexagonal

```
Presentation (API + Frontend)
    ↓ depends on
Application (Use Cases, Services)
    ↓ depends on
Domain (Entities, Interfaces)
    ↑ implemented by
Infrastructure (Repositories, Feedback Providers)
```

The dependency inversion principle ensures the domain has zero knowledge of infrastructure, making the domain model the source of truth for the assignment's LLD exercise.

Key Trade-offs

Decision	Trade-off	Rationale
In-memory storage	No persistence across restarts	Simplicity; easily swappable via repository interface
Both providers run (merge)	Slower than one-provider	RicHER feedback; LLM can fail but rules still provide baseline
Simulated LLM mode	Not "real" LLM feedback	Keeps prototype self-contained; real API is plug-and-play
Monolith	Not horizontally scalable	Assignment explicitly states a simple monolith is acceptable
Deterministic rules + LLM simulation	Heuristic rules are imperfect	Provides reproducible baseline; LLM adds depth when available